



UNIVERSIDAD AUTONOMA DE
ZACATECAS

ANALISIS Y DISEÑO DE ALGORITMOS

Reporte del rendimiento de algoritmos

Alumno: Devani Carolina Trejo Martinez

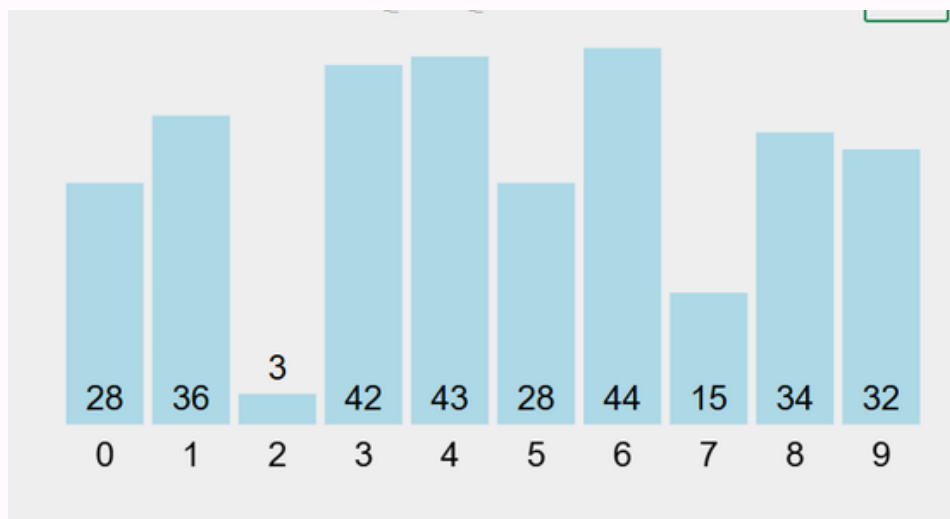
Profesor: Alejandro Morgan

MARZO 2026

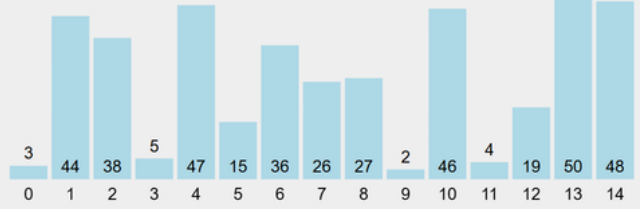
MÉTODOS DE ORDENAMIENTO

Título del Proyecto: Análisis de Algoritmos

A continuación, en este documento se hablará acerca de los órdenes de crecimiento como BubbleSort, Búsqueda binaria, MergeSort, InsertSort, QuickSort, SelectionSort, y se harán pruebas de rendimiento para analizar cuál se tarda más y cuál se tarda menos con la ayuda de los algoritmos de VISUALGO.



Bubble Sort



```
def bubble_sort(lista):
```

```
    swap_counter = 0
```

```
    n = len(lista)
```

```
    swapped = True
```

```
    while swapped:
```

```
        swapped = False
```

```
        for i in range(0, n - 1):
```

```
            if lista[i] > lista[i + 1]:
```

```
                # Intercambio
```

```
                lista[i], lista[i + 1] = lista[i + 1], lista[i]
```

```
                swapped = True
```

```
                swap_counter += 1
```

```
        n -= 1 # Reduce el rango de comparación
```

```
    return lista, swap_counter
```

```
# Ejemplo de uso
```

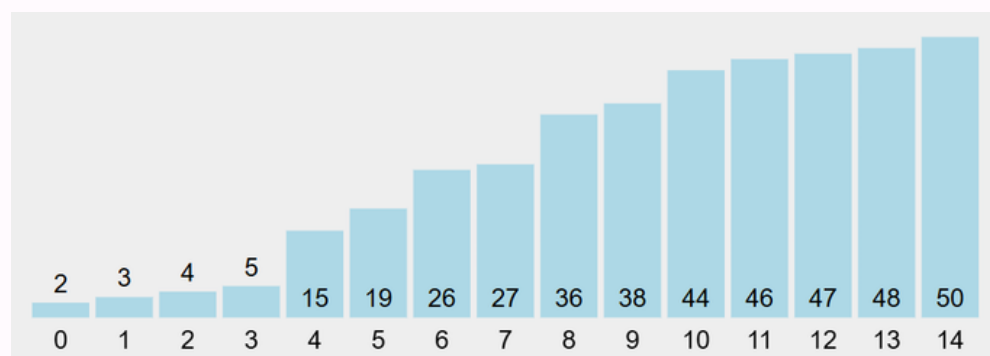
```
datos = [7, 2, 5, 1, 9]
```

```
ordenada, intercambios = bubble_sort(datos)
```

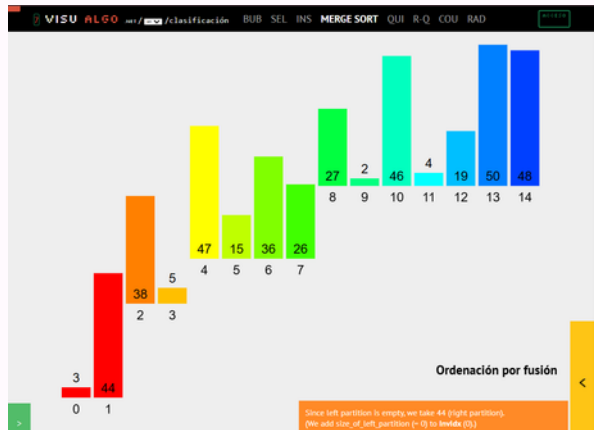
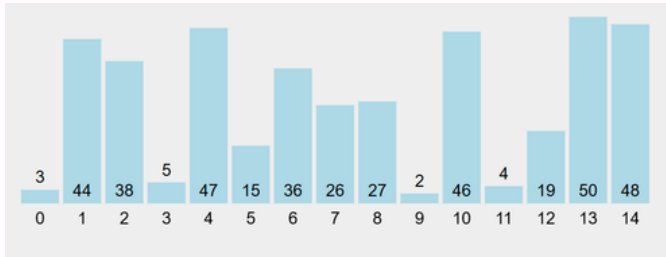
```
print("Lista ordenada:", ordenada)
```

```
print("Intercambios:", intercambios)
```

El ordenamiento de Bubble Sort es el más sencillo y funciona intercambiando repetidamente los elementos adyacentes si están en el orden incorrecto. Este algoritmo no es suficiente para grandes conjuntos de datos, por su complejidad temporal, tanto en el caso promedio como en el peor, es bastante alta, ya que su complejidad es $O(n^2)$.



Merge Sort



```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)
```

```
i = j = k = 0
```

```
while i < len(L) and j < len(R):
```

```
    if L[i] < R[j]:
```

```
        arr[k] = L[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = R[j]
```

```
        j += 1
```

```
    k += 1
```

```
while i < len(L):
```

```
    arr[k] = L[i]
```

```
    i += 1
```

```
    k += 1
```

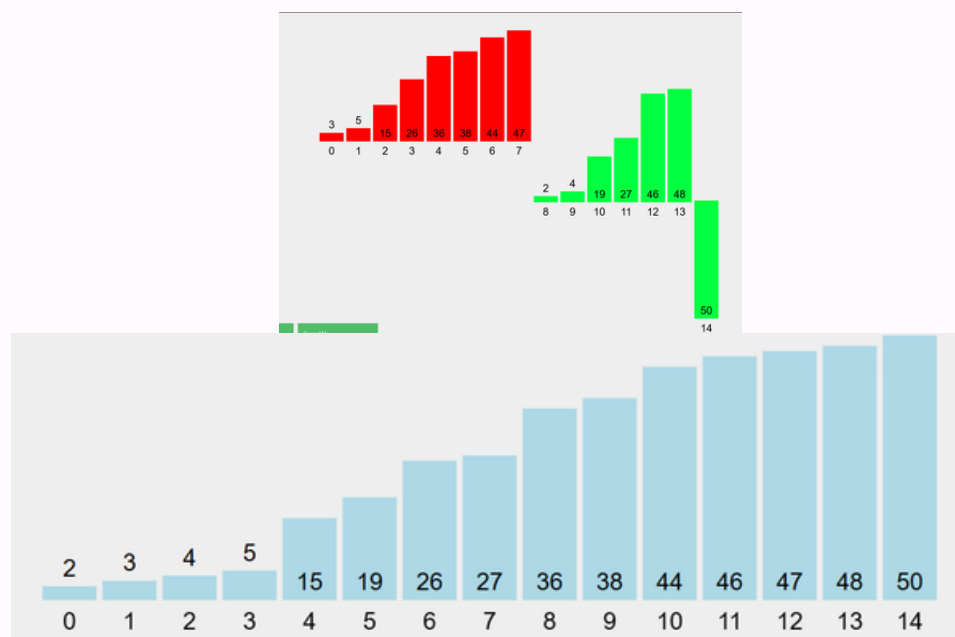
```
while j < len(R):
```

```
    arr[k] = R[j]
```

```
    j += 1
```

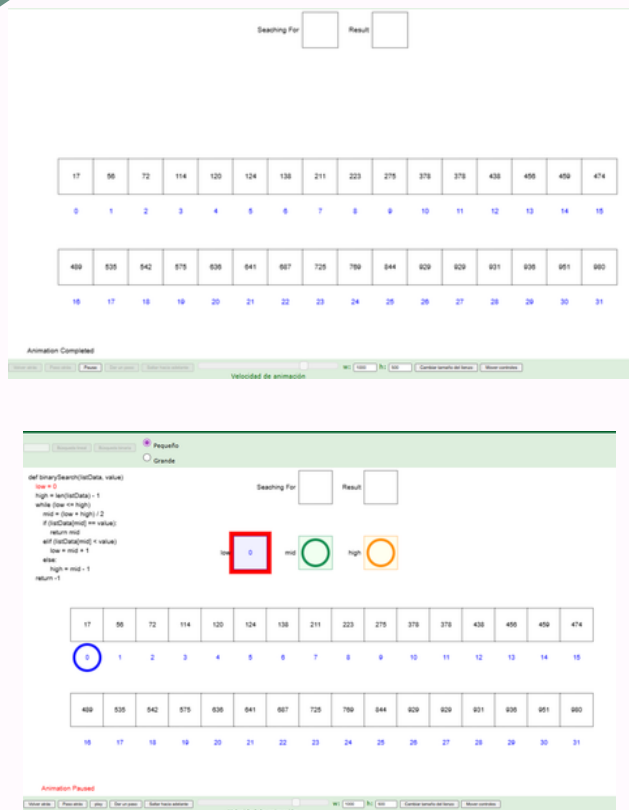
```
    k += 1
```

El algoritmo de ordenación Mergesort es un método eficiente, general y comparativo para ordenar listas o arrays. Es un ejemplo clásico de un algoritmo "divide y vencerás" mergesort divide el conjunto de datos en mitades más pequeñas las ordena y luego la fusiona de nuevo en una sola lista ordenada. Su complejidad temporal pertenece a $O(n \log n)$



MARZO 2026

Búsqueda binaria



```
def binary_search(listData, value):
```

```
    low = 0
```

```
    high = len(listData) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2 # División entera
```

```
        if listData[mid] == value:
```

```
            return mid
```

```
        elif listData[mid] < value:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

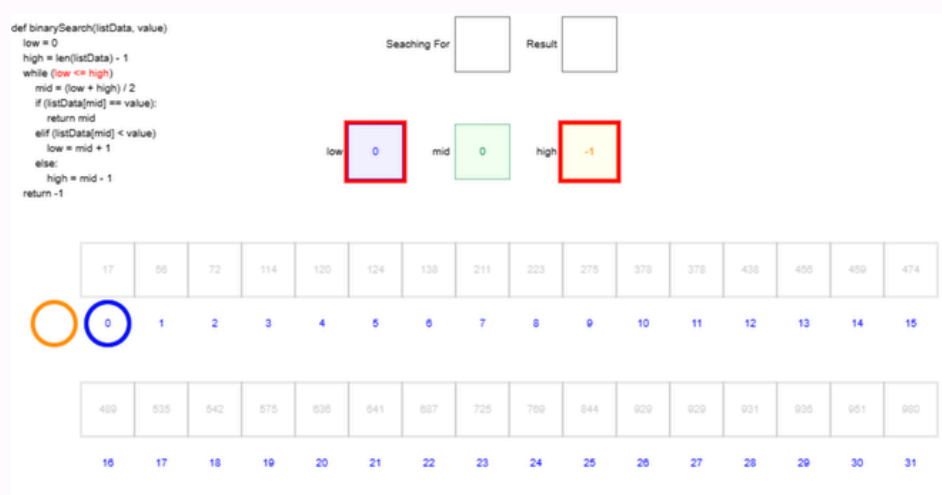
Ejemplo de uso

```
datos = [1, 3, 5, 7, 9, 11]
```

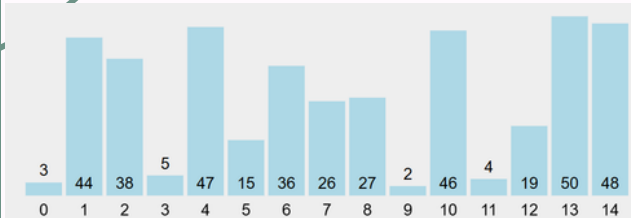
```
resultado = binary_search(datos, 7)
```

```
print("Índice encontrado:", resultado)
```

La búsqueda binaria es un algoritmo eficiente para encontrar un elemento en una lista ordenada de elementos funciona al dividir repetidamente a la mitad la porción de la lista que podría contener al elemento hasta reducir las ubicaciones posibles a una sola. Este método de ordenamiento tiene un crecimiento de $O(\log n)$.



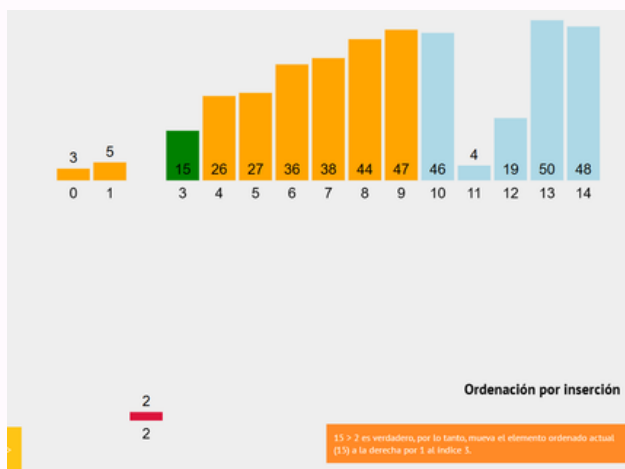
Insert Sort



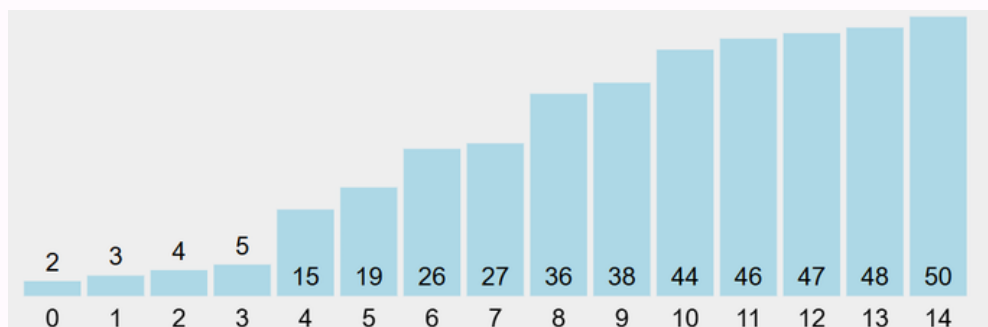
```
mylist = [64, 34, 25, 12, 22, 11, 90, 5]
```

```
n = len(mylist)
for i in range(1,n):
    insert_index = i
    current_value = mylist.pop(i)
    for j in range(i-1, -1, -1):
        if mylist[j] > current_value:
            insert_index = j
    mylist.insert(insert_index,
current_value)
```

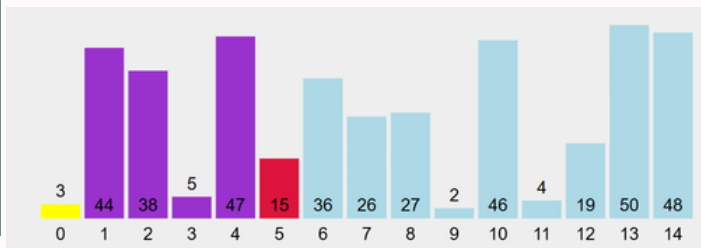
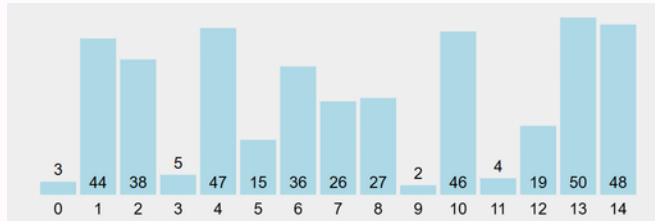
```
print(mylist)
```



El algoritmo de ordenación por inserción utiliza una parte del array para almacenar los valores ordenados y la otra parte para almacenar los valores que aún no están ordenados. El algoritmo toma un valor a la vez de la parte no ordenada del array y lo coloca en el lugar correcto en la parte ordenada del array, hasta que el array esté ordenado., es de complejidad $O(n^2)$



Quiker Sort

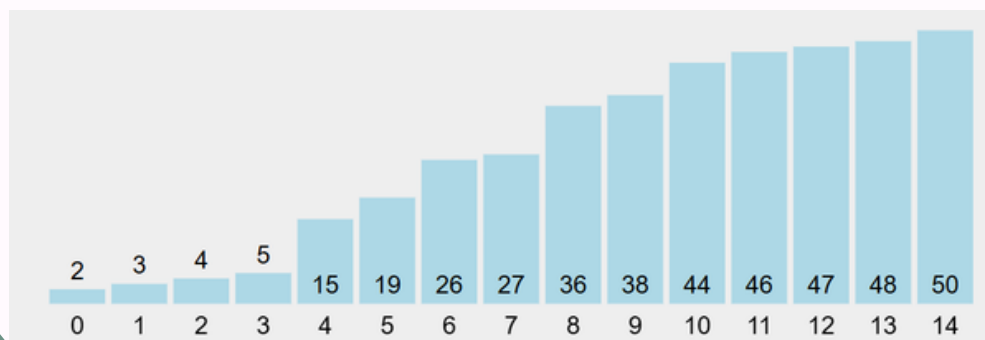


```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

```
def quick_sort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
```

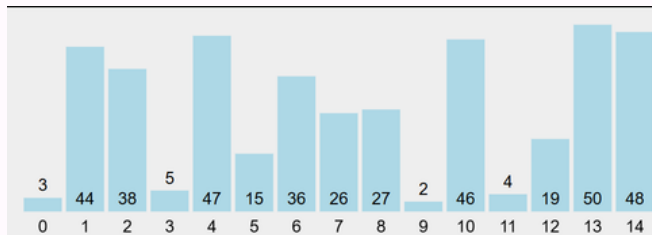
```
arr = [1, 7, 4, 1, 10, 9, -2]
quick_sort(arr, 0, len(arr) - 1)
print(arr)
```

QuickSort es un algoritmo de ordenación basado en el principio de "divide y vencerás" que selecciona un elemento como pivote y divide el array dado alrededor del pivote seleccionado, colocando este último en la posición correcta dentro del array ordenado.. Este tiene complejidad $O(n \log n)$, lo que lo hace muy eficiente para grandes conjuntos de datos



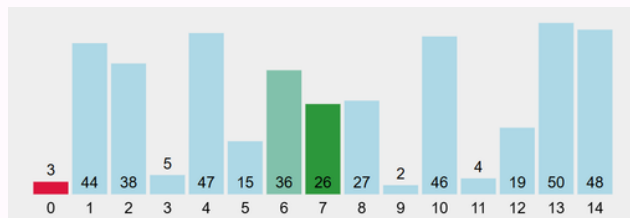
Selecton Sort

```
mylist = [64, 34, 25, 5, 22, 11, 90, 12]
```

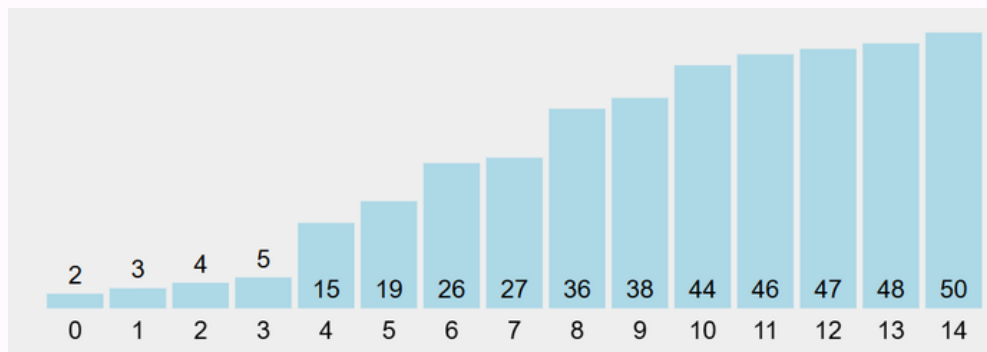


```
n = len(mylist)
for i in range(n-1):
    min_index = i
    for j in range(i+1, n):
        if mylist[j] < mylist[min_index]:
            min_index = j
    min_value = mylist.pop(min_index)
    mylist.insert(i, min_value)
```

```
print(mylist)
```



Selection Sort es un algoritmo de ordenamiento que divide el arreglo en dos partes: una parte ordenada que se construye gradualmente y otra no ordenada. En cada iteración, encuentra el elemento más pequeño de la parte no ordenada y lo coloca al inicio de la parte ordenada.. Su complejidad es $O(n^2)$ lo que lo hace ineficiente para las listas grandes



Conclusion

En conclusión, QuickSort y MergeSort son, en general, los mejores métodos de ordenamiento por su eficiencia en conjuntos de datos grandes, siendo QuickSort más rápido en promedio y MergeSort más estable y predecible. Para datos pequeños o casi ordenados, Insertion Sort es superior, mientras que Bubble Sort es el menos eficiente.

- QuickSort: Es generalmente el más rápido en la práctica para la mayoría de los propósitos, utilizando un enfoque de "divide y vencerás".
- MergeSort: Ofrece un rendimiento garantizado incluso en el peor caso y es estable, ideal para estructuras de datos que requieren ordenamiento secuencial.
- Insertion Sort: Es muy eficiente para conjuntos de datos muy pequeños o listas casi ordenadas.
- Selection Sort: Es sencillo pero ineficiente, superior solo a Bubble Sort en pocos casos, ya que reduce el número de intercambios.
- Bubble Sort: Es el algoritmo más lento y menos eficiente, utilizado principalmente con fines educativos.
- Búsqueda binaria es rápida siempre y cuando solo busque los elementos, ya que no es un método de ordenamiento como tal.